network. In this section we'll delve into how to actually implement these algorithms and train (i.e., optimize) them.

4.2.1. Defining an objective

Recall that neural networks need an objective function that is differentiable with respect to the network weights (parameters). In the last chapter we trained the deep Q-network with a minimizing mean squared error (MSE) loss function with respect to its predicted Q values and the target Q value. We had a nice formula for calculating the target Q value based on the observed reward, since Q values are just averaged rewards (i.e., expectations), so this was not much different from how we would normally train a supervised deep learning algorithm.

How do we train a policy network that gives us a probability distribution over actions given a state, $P(A|S)$? There's no obvious way to map our observed rewards after taking an action to updating $P(A|S)$. Training the DQN was not much different from solving a supervised learning problem because our Q-network generated a vector of predicted Q values, and by using a formula we were able to generate the target Q value vector. Then we just minimized the error between the Q-network's output vector and our target vector.

With a policy network, we're predicting actions directly, and there is no way to come up with a target vector of actions we should have taken instead, given the rewards. All we know is whether the action led to positive or negative rewards. In fact, what the best action is secretly depends on a value function, but with a policy network we're trying to avoid computing these action values directly.

Let's go through an example to see how we might optimize our policy network. We'll start with some notation. Our policy network is denoted π and is parameterized by a vector θ, which represents all of the parameters (weights) of the neural network. As you know, neural networks have parameters in the form of multiple weight matrices, but for the purposes of